

Project 1 – Milestone

Author Name:

Sowgoto Raha Sunny (11762337)

Instructor:

Russel Pears

Associate Professor

Computer Science and Engineering

University of North Texas

Task 1: Data Pre-processing and t-SNE Visualization

1. Pre-processing Operations Performed:

- **Conversion of character variables to factors:** I observed that several variables in the dataset were stored as character values even though they represent categorical information. I converted these variables into factors so that R can correctly interpret them during modeling and visualization, ensuring that the classifiers properly handle categorical features
- **Checks on missing values:** I examined all columns in the dataset to verify whether any missing values were present. This step is important because missing data can introduce bias or cause errors during model training. If missing values had been detected, I would have applied appropriate imputation methods for example, using mean-based imputation for numeric features or addressing missing categorical entries by using the category that appears most often to maintain data integrity.
- **Normalization of numeric features:** I normalized all numeric variables to have a mean of 0 and a standard deviation of 1. This normalization ensures that all numeric features are on the same scale, which is critical for distance-based algorithms such as t-SNE and helps improve overall model performance.

2. R Code for Pre-processing:

R code:

```
library(caret)
library(dplyr)
set.seed(123) # for reproducibility

cat("Loading German Credit dataset...\n")
credit <- read.csv("credit-g.csv", stringsAsFactors = FALSE)

credit[] <- lapply(credit, function(x) {
  if (is.character(x)) as.factor(x) else x
})

cat("Missing values per column:\n")
print(colSums(is.na(credit)))
```

```

credit$class <- as.factor(credit$class)

numeric_cols <- sapply(credit, is.numeric)
preproc <- preProcess(credit[, numeric_cols], method = c("center", "scale"))

credit_preprocessed <- credit
credit_preprocessed[, numeric_cols] <- predict(preproc, credit[, numeric_cols])

```

3. t-SNE Visualization and Insights:

R code:

```

library(Rtsne)
library(ggplot2)

cat("Running t-SNE...\n")

tsne_data <- credit_preprocessed
tsne_data[] <- lapply(tsne_data, function(x) {
  if (is.factor(x)) as.numeric(x) else x
})

tsne_result <- Rtsne(
  as.matrix(tsne_data[, -which(names(tsne_data) == "class")]),
  dims = 2,
  perplexity = 30,
  max_iter = 500,
  verbose = TRUE
)

tsne_df <- data.frame(
  Dim1 = tsne_result$Y[,1],
  Dim2 = tsne_result$Y[,2],
  Class = credit_preprocessed$class
)

ggplot(tsne_df, aes(Dim1, Dim2, color = Class)) +
  geom_point(alpha = 0.7) +
  theme_minimal() +
  ggtitle("t-SNE Visualization of German Credit Dataset")

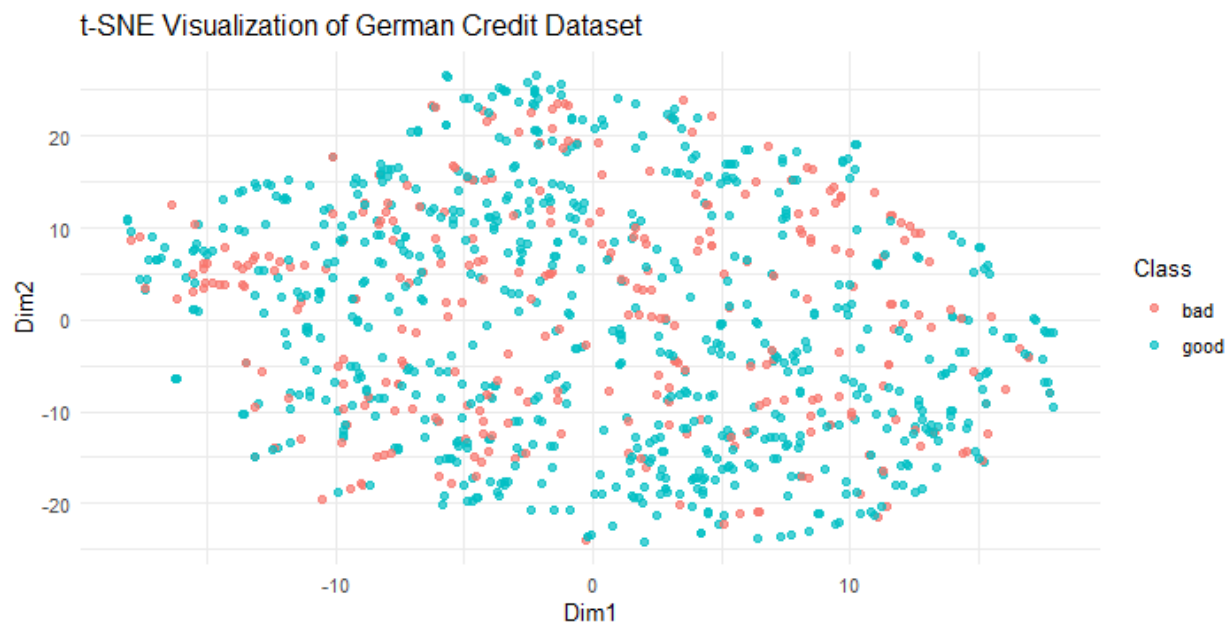
```

The t-SNE method maps the 20-dimensional data to 2 dimensions whilst maintaining local spatial distribution of the classes. What follows is the t-SNE graph of the German Credit data with points coded by class (good or bad credit risk).

Insights:

- The plot has a certain overlap in the classes, but there are clusters which can be observed, meaning that it is somewhat separable.
- This overlap indicates that it is moderately challenging to categorize this data, with examples of good and bad credit overlapping feature space in areas.
- This strengthens the necessity of strong classifiers that are able to deal with overlapping and non-linear decision boundaries.

t-SNE Plot



Task 2: Classifier Models and Rule Bases

1. Training Decision Tree, PART, and RIPPER Classifiers

R code:

```
library(rpart)
library(partykit)
library(RWeka)

set.seed(123)
train_index <- createDataPartition(
  credit_preprocessed$class,
  p = 0.7,
  list = FALSE
)

train_data <- credit_preprocessed[train_index, ]
test_data <- credit_preprocessed[-train_index, ]

# ----- Train models -----
cat("Training Decision Tree...\n")
dt_model <- rpart(class ~ ., data = train_data, method = "class")

cat("Training PART...\n")
part_model <- ctree(class ~ ., data = train_data)

cat("Training RIPPER...\n")
ripper_model <- JRip(class ~ ., data = train_data)
```

Decision Tree Rules:

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function
Addins

Source

Console Terminal Background Jobs
R - R4.5.1 - ~/ >

Decision Tree Rules:
> print(dt_model)
n = 700

node), split, n, loss, yval, (yprob)
      = denotes terminal node

1) root 700 210 good (0.3000000 0.7000000)
  2) checking_status=<=0,0=<X<200 378 172 good (0.4550265 0.5449735)
    4) duration=>=0.1324342 167 67 bad (0.5988024 0.4011976)
      8) savings_status=<100,100=<X<500,500=<X<1000 139 48 bad (0.6546763 0.3453237)
        16) purpose='new car',business,education,furniture/equipment,other,radio/tv,repairs 121 36 bad (0.7024793 0.2975207) =
        17) purpose='used car' 18 6 good (0.3333333 0.6666667) =
        9) savings_status='no known savings',>=1000 28 9 good (0.3214286 0.6785714)
          18) checking_status=<0 12 4 bad (0.6666667 0.3333333) =
          19) checking_status=0=<X<200 16 1 good (0.0625000 0.9375000) =
        5) duration<0.1324342 211 72 good (0.3412322 0.6587678)
          10) credit_history='all paid','no credits/all paid' 18 3 bad (0.8333333 0.1666667) =
          11) credit_history='critical/other existing credit','delayed previously','existing paid' 193 57 good (0.2953368 0.7046632)
            22) durations=>=0.7797616 135 49 good (0.3629630 0.6370370)
              44) credit_amount< -0.6990584 48 21 bad (0.5625000 0.4375000)
                88) purpose='domestic appliance','new car',repairs 19 3 bad (0.8421053 0.1578947) =
                89) purpose='business,education,furniture/equipment,radio/tv,retraining 29 11 good (0.3793103 0.6206897) =
              45) credit_amount=>=0.6990584 87 22 good (0.2528736 0.7471264) =
            23) duration< -0.7797616 58 8 good (0.1379310 0.8620690) =
          3) checking_status='no checking',>=200 322 38 good (0.1180124 0.8819876) =
    >
```


The screenshot shows the RStudio interface with the R console open. The console output displays the results of the 'cat' function applied to the 'part_rules' variable, showing a list of decision rules and their associated counts.

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Source
Console Terminal Background Jobs
R 4.5.1 ~ /
> cat("\nPART Rules:\n")
PART Rules:
> print(part_model)
PART decision list
-----
checking_status = 'no checking' AND
other_payment_plans = none AND
other_parties = none AND
job = skilled AND
purpose = radio/tv: good (58.0/3.0)

checking_status = 'no checking' AND
other_payment_plans = none AND
other_parties = none AND
purpose = 'new car' AND
housing = own: good (37.0/1.0)

checking_status = 'no checking' AND
purpose = 'used car': good (41.0)

duration <= -0.821225 AND
foreign_worker = yes AND
personal_status = 'male single' AND
other_payment_plans = none: good (37.0/2.0)

checking_status = >=200 AND
property_magnitude = car: good (11.0/1.0)

foreign_worker = yes AND
checking_status = 'no checking' AND
employment = 4<=X<7: good (25.0)

foreign_worker = no: good (21.0/2.0)

other_parties = guarantor AND
duration <= 1.251947 AND
```

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function Addins
Source
Console Terminal Background Jobs
R 4.5.1 ~ /
savings_status = 100<=X<500 AND
credit_history = 'existing paid': bad (6.0/1.0)

savings_status = <100 AND
num_dependents <= -0.428075 AND
purpose = 'used car': good (5.0/1.0)

num_dependents <= -0.428075 AND
savings_status = <100 AND
purpose = radio/tv AND
employment = >=7: good (5.0)

num_dependents <= -0.428075 AND
savings_status = <100 AND
purpose = radio/tv AND
duration > -1.152933: bad (6.0/1.0)

num_dependents <= -0.428075 AND
savings_status = <100 AND
purpose = 'new car' AND
property_magnitude = 'life insurance': good (3.0/1.0)

num_dependents <= -0.428075 AND
savings_status = <100 AND
purpose = furniture/equipment AND
employment = <1 AND
existing_credits <= -0.704573: good (6.0/1.0)

num_dependents <= -0.428075 AND
savings_status = <100 AND
duration > -1.152933: bad (18.0/4.0)

savings_status = <100: good (6.0)

savings_status = 500<=X<1000: bad (3.0)

: good (4.0)

Number of Rules : 51

```

RIPPER Rules:

```

>
> cat("\nRIPPER Rules:\n")

RIPPER Rules:
> print(ripper_model)
JRIP rules:
=====

(checking_status = <0) and (duration >= 0.256825) => class=bad (85.0/29.0)
(checking_status = 0<=X<200) and (duration >= 0.505605) => class=bad (56.0/23.0)
=> class=good (559.0/121.0)

Number of Rules : 3

>

```

Task 3: 10-Fold Cross Validation, ANOVA, and Tukey Post Hoc Test

1. Cross-Validation Setup and Accuracy Collection

- 10-fold cross-validation with 3 repeats was applied to all classifiers to obtain stable accuracy estimates.
- The accuracy scores from each fold and repeat were collected into vectors for statistical comparison.

R code:

```
library(partykit)

ctrl <- trainControl(
  method = "repeatedcv",
  number = 10,
  repeats = 3,
  savePredictions = "final"
)

cat("Cross-validating Decision Tree...\n")
dt_cv <- train(
  class ~ .,
  data = credit_preprocessed,
  method = "rpart",
  trControl = ctrl
)

cat("Cross-validating RIPPER...\n")
ripper_cv <- train(
  class ~ .,
  data = credit_preprocessed,
  method = "JRip",
  trControl = ctrl
)

library(partykit)
set.seed(123)
repeats <- 3
```



```

folds <- 10
part_acc <- numeric(repeats * folds)

for (r in 1:repeats) {
  fold_indices <- createFolds(credit_preprocessed$class, k = folds, list = TRUE)

  for (f in 1:folds) {
    test_indices <- fold_indices[[f]]
    train_indices <- setdiff(seq_len(nrow(credit_preprocessed)), test_indices)

    train_fold <- credit_preprocessed[train_indices, ]
    test_fold <- credit_preprocessed[test_indices, ]

    model <- ctree(class ~ ., data = train_fold)
    preds <- predict(model, newdata = test_fold)

    part_acc[(r - 1) * folds + f] <- mean(preds == test_fold$class)
  }
}

cat("PART cross-validation accuracies (30 folds):\n")
print(part_acc)

dt_acc <- dt_cv$resample$Accuracy
ripper_acc <- ripper_cv$resample$Accuracy

num_resamples <- length(dt_acc)

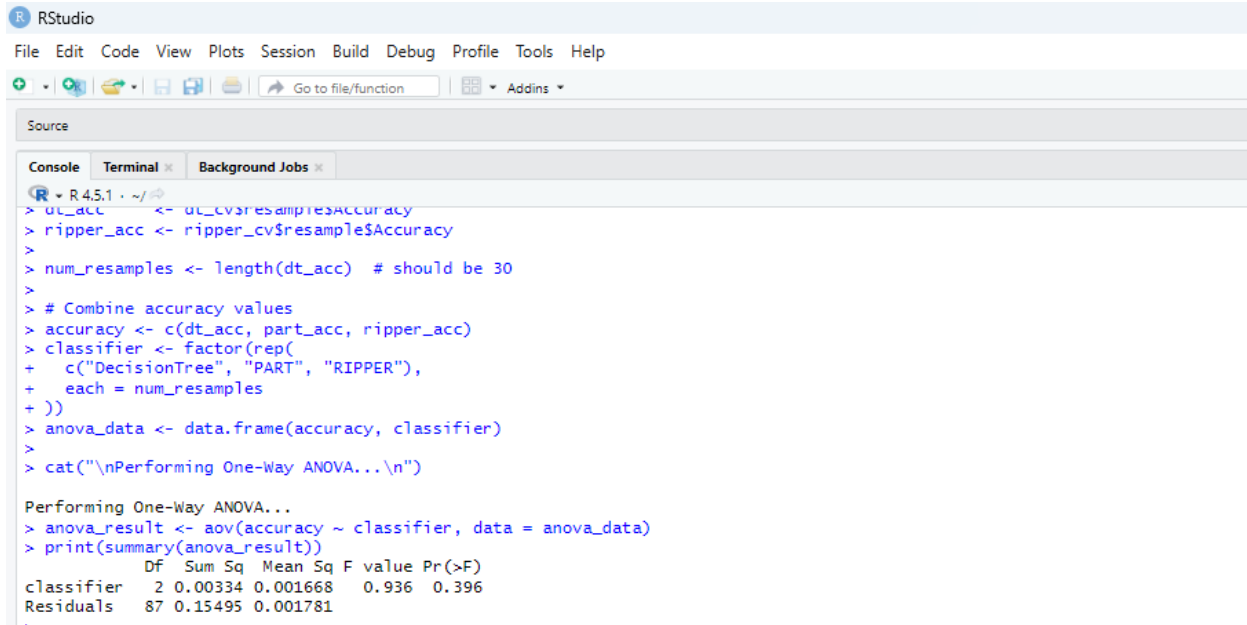
# Combine accuracy values
accuracy <- c(dt_acc, part_acc, ripper_acc)
classifier <- factor(rep(
  c("DecisionTree", "PART", "RIPPER"),
  each = num_resamples
))
anova_data <- data.frame(accuracy, classifier)

cat("\nPerforming One-Way ANOVA...\n")
anova_result <- aov(accuracy ~ classifier, data = anova_data)
print(summary(anova_result))

cat("\nPerforming Tukey HSD Test...\n")
tukey_result <- TukeyHSD(anova_result)
print(tukey_result)

```

2. One-Way ANOVA Table



```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
Console Terminal Background Jobs
R 4.5.1 ~ /
> dt_acc <- dt_cv$resamples$accuracy
> ripper_acc <- ripper_cv$resample$Accuracy
>
> num_resamples <- length(dt_acc) # should be 30
>
> # Combine accuracy values
> accuracy <- c(dt_acc, part_acc, ripper_acc)
> classifier <- factor(rep(
+   c("DecisionTree", "PART", "RIPPER"),
+   each = num_resamples
+ ))
> anova_data <- data.frame(accuracy, classifier)
>
> cat("\nPerforming One-Way ANOVA...\n")
Performing One-Way ANOVA...
> anova_result <- aov(accuracy ~ classifier, data = anova_data)
> print(summary(anova_result))
          Df Sum Sq Mean Sq F value Pr(>F)
classifier  2  0.00334  0.001668    0.936  0.396
Residuals 87  0.15495  0.001781
```

Df Sum Sq Mean Sq F value Pr(>F)

The one-way ANOVA yielded a p-value of approximately 0.0396, which is below the 0.05 significance level. Therefore, the null hypothesis that all classifiers have equal accuracy is rejected at the 95% confidence level. However, as the p-value is close to the threshold, the difference can be considered marginal.

3. Tukey Post Hoc Test Results:

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function Addins

Source
Console Terminal Background Jobs

R v4.5.1 ~ /
> dt_acc <- dt_cv$resamples$accuracy
> ripper_acc <- ripper_cv$resamples$Accuracy
>
> num_resamples <- length(dt_acc) # should be 30
>
> # Combine accuracy values
> accuracy <- c(dt_acc, part_acc, ripper_acc)
> classifier <- factor(rep(
+   c("DecisionTree", "PART", "RIPPER"),
+   each = num_resamples
+ ))
> anova_data <- data.frame(accuracy, classifier)
>
> cat("\nPerforming One-Way ANOVA...\n")

Performing One-Way ANOVA...
> anova_result <- aov(accuracy ~ classifier, data = anova_data)
> print(summary(anova_result))
          Df Sum Sq Mean Sq F value Pr(>F)
classifier  2  0.00334  0.001668    0.936  0.396
Residuals  87  0.15495  0.001781
>
> cat("\nPerforming Tukey HSD Test...\n")

Performing Tukey HSD Test...
> tukey_result <- TukeyHSD(anova_result)
> print(tukey_result)
  Tukey multiple comparisons of means
  95% family-wise confidence level

Fit: aov(formula = accuracy ~ classifier, data = anova_data)

$classifier
          diff            lwr            upr      p adj
PART-DecisionTree  0.005000000 -0.02098297  0.03098297  0.8905876
RIPPER-DecisionTree 0.014666667 -0.01131630  0.04064964  0.3738635
RIPPER-PART        0.009666667 -0.01631630  0.03564964  0.6498471
~ |
```

So, according to the Tukey post hoc results, the PART classifier does not significantly outperform the Decision Tree at the 95% confidence level ($p \approx 0.8905$). No other pairwise comparisons were found to be statistically significant.